BUSINESS CASE STUDY: TARGET

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1. Data type of all columns in the "customers" table.

SQL CODE :

```
SELECT
  customer_id,
  customer_unique_id,
  customer_zip_code_prefix,
  customer_city,
  customer_state
FROM`Target.customers`
LIMIT10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_id	customer_unique_id	customer_zip_co...	customer_city	customer_state
1	2201362e68992f654942dc006...	f7d7fc0a59ef4363fdce6e3aa06...	69900	rio branco	AC
2	31dbc13addc753e210692eaca...	5dbba6c01268a8ad43f79157bf...	69900	rio branco	AC
3	dad907e170748a35ef4e92238...	36b1c0516f123351ffa8743041...	69900	rio branco	AC
4	888d2ebe1af2a8c93c75dae5df...	721d1092e1a6460c67e6a0e69...	69900	rio branco	AC
5	8a0108267d9258a0ec9f74381...	7a2dc4682890550ebe3b8befce...	69900	rio branco	AC
6	5880e46677c68394bda62479f...	5dbba6c01268a8ad43f79157bf...	69900	rio branco	AC
7	53996870173a3a001a1fb56ef0...	2624230437101e0bdcea1e483...	69900	rio branco	AC
8	cd281c1a7d26cd29a3ed4b029f...	086d6b5b5ba195a91aa0a6ec8...	69900	rio branco	AC
9	717a3a9459f9006f23e24a684b...	5203034a29c3d294713bf8ccb...	69900	rio branco	AC
10	2d618e470c95c9b425cbb0cbc...	f7d7fc0a59ef4363fdce6e3aa06...	69900	rio branco	AC

2.Get the time range between which the orders were placed.

SQL CODE :

```
SELECT
  MIN(order_purchase_timestamp)ASfirst_order_date,
  MAX(order_purchase_timestamp)ASlast_order_date
FROM`Target.orders`;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	first_order_date	last_order_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

3. Count the Cities & States of customers who ordered during the given period.

SQL CODE :
```
SELECT
  COUNT(DISTINCTcustomer_city)ASunique_customer_cities,
  COUNT(DISTINCTcustomer_state)ASunique_customer_states
  FROM`Target.customers`;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	unique_customer...	unique_customer...
1	4119	27

2.In-depth Exploration:

1. Is there a growing trend in the number of orders placed over the past years?

SQL CODE :
```
SELECT
  EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
  COUNT(order_id) AS total_orders
FROM `Target.orders`
GROUP BY order_year
ORDER BY order_year;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	order_year	total_orders
1	2016	329
2	2017	45101
3	2018	54011

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

SQL CODE : SELECT

```
  EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
  EXTRACT(MONTH FROM order_purchase_timestamp) AS
order_month,
  COUNT(order_id) AS total_orders
FROM `Target.orders`
GROUP BY order_year, order_month
ORDER BY order_year, order_month
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

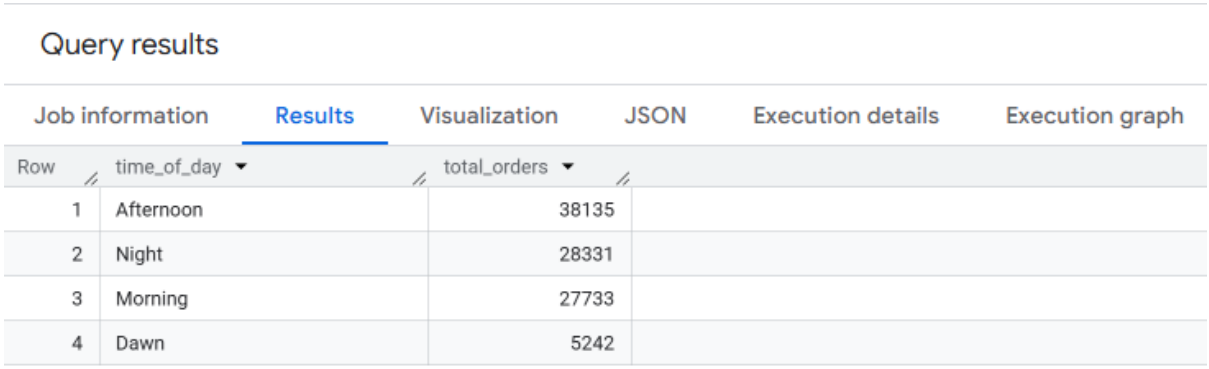
Row	order_year	order_month	total_orders
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700
9	2017	6	3245
10	2017	7	4026

3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
0-6 hrs : Dawn
7-12 hrs : Mornings
13-18 hrs : Afternoon
19-23 hrs : Night

SQL CODE :

```
SELECT
  CASE
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN
0 AND 6 THEN 'Dawn'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN
7 AND 12 THEN 'Morning'
```

```
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN
13 AND 18 THEN 'Afternoon'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN
19 AND 23 THEN 'Night'
  END AS time_of_day,
  COUNT(order_id) AS total_orders
FROM `Target.orders`
GROUP BY time_of_day
ORDER BY total_orders DESC;
```



Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	time_of_day	total_orders
1	Afternoon	38135
2	Night	28331
3	Morning	27733
4	Dawn	5242

3. Evolution of E-commerce orders in the Brazil region:

1. Get the month on month no. of orders placed in each state.

SQL CODE :

```
SELECT
  EXTRACT(YEAR FROM o.order_purchase_timestamp) AS
order_year,
  EXTRACT(MONTH FROM o.order_purchase_timestamp) AS
order_month,
  c.customer_state,
  COUNT(o.order_id) AS total_orders
FROM `Target.orders` o
JOIN `Target.customers` c
  ON o.customer_id = c.customer_id
GROUP BY order_year, order_month, c.customer_state
ORDER BY order_year, order_month, total_orders DESC
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	order_year	order_month	customer_state	total_orders
1	2016	9	SP	2
2	2016	9	RS	1
3	2016	9	RR	1
4	2016	10	SP	113
5	2016	10	RJ	56
6	2016	10	MG	40
7	2016	10	RS	24
8	2016	10	PR	19
9	2016	10	SC	11
10	2016	10	GO	9

2. How are the customers distributed across all the states?

SQL CODE :

```
SELECT
  c.customer_state,
  COUNT(DISTINCT c.customer_id) AS total_customers
FROM `Target.customers` c
GROUP BY c.customer_state
ORDER BY total_customers DESC
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	total_customers
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

4.Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

SQL CODE :

```
WITH yearly_cost AS (
  SELECT
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS
order_year,
    SUM(p.payment_value) AS total_payment
  FROM `Target.orders` o
  JOIN `Target.payments` p
    ON o.order_id = p.order_id
  WHERE EXTRACT(MONTH FROM o.order_purchase_timestamp)
BETWEEN 1 AND 8
  GROUP BY order_year
)
SELECT
  MAX(CASE WHEN order_year = 2017 THEN total_payment END) AS
total_2017,
```

```
  MAX(CASE WHEN order_year = 2018 THEN total_payment END) AS
total_2018,
  ROUND( ( (MAX(CASE WHEN order_year = 2018 THEN
total_payment END) -
             MAX(CASE WHEN order_year = 2017 THEN
total_payment END)) /
             MAX(CASE WHEN order_year = 2017 THEN
total_payment END) ) * 100 , 2
  ) AS percent_increase
FROM yearly_cost;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	total_2017	total_2018	percent_increase
1	3669022.1200000085	8694733.8400000166	136.98

2. Calculate the Total & Average value of order price for each state.

SQL CODE :

```
SELECT
  c.customer_state,
  ROUND(SUM(oi.price), 2) AS total_order_price,
  ROUND(AVG(oi.price), 2) AS avg_order_price
FROM `Target.order_items` oi
JOIN `Target.orders` o
  ON oi.order_id = o.order_id
JOIN `Target.customers` c
  ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY total_order_price DESC
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	total_order_price	avg_order_price
1	SP	5202955.05	109.65
2	RJ	1824092.67	125.12
3	MG	1585308.03	120.75
4	RS	750304.02	120.34
5	PR	683083.76	119.0
6	SC	520553.34	124.65
7	BA	511349.99	134.6
8	DF	302603.94	125.77
9	GO	294591.95	126.27
10	ES	275037.31	121.91

3. Calculate the Total & Average value of order freight for each state.

SQL CODE :

```
SELECT
  c.customer_state,
  ROUND(SUM(oi.freight_value), 2) AS total_freight_value,
  ROUND(AVG(oi.freight_value), 2) AS avg_freight_value
FROM `Target.order_items` oi
JOIN `Target.orders` o
  ON oi.order_id = o.order_id
JOIN `Target.customers` c
  ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY total_freight_value DESC
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	customer_state	total_freight_value	avg_freight_value
1	SP	718723.07	15.15
2	RJ	305589.31	20.96
3	MG	270853.46	20.63
4	RS	135522.74	21.74
5	PR	117851.68	20.53
6	BA	100156.68	26.36
7	SC	89660.26	21.47
8	PE	59449.66	32.92
9	GO	53114.98	22.77
10	DF	50625.5	21.04

4. Analysis based on sales, freight and delivery time.

1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

SQL CODE :

```
SELECT
  order_id,
  DATE(order_purchase_timestamp) AS purchase_date,
  DATE(order_delivered_customer_date) AS delivered_date,
  DATE(order_estimated_delivery_date) AS estimated_date,

  DATE_DIFF(order_delivered_customer_date,
order_purchase_timestamp, DAY) AS actual_delivery_days,

  DATE_DIFF(order_delivered_customer_date,
order_estimated_delivery_date, DAY) AS
diff_estimated_vs_actual
FROM `Target.orders`
WHERE order_delivered_customer_date IS NOT NULL
  AND order_estimated_delivery_date IS NOT NULL
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	order_id	purchase_date	delivered_date	estimated_date	actual_delivery_d...	diff_estimated_vs...
1	65d1e226dfaeb8cdc42f665422...	2016-10-03	2016-11-08	2016-11-25	35	-16
2	770d331c84e5b214bd9dc70a1...	2016-10-07	2016-10-14	2016-11-29	7	-45
3	dabf2b0e35b423f94618bf965fc...	2016-10-09	2016-10-16	2016-11-30	7	-44
4	8beb59392e21af5eb9547ae1a9...	2016-10-08	2016-10-19	2016-11-30	10	-41
5	2c45c33d2f9cb8ff8b1c86cc28...	2016-10-09	2016-11-09	2016-12-08	30	-28
6	1950d777989f6a877539f53795...	2018-02-19	2018-03-21	2018-03-09	30	12
7	bfbd0f9bdef84302105ad712db...	2016-09-15	2016-11-09	2016-10-04	54	36
8	3b697a20d9e427646d9256791...	2016-10-03	2016-10-26	2016-10-27	23	0
9	be5bc2f0da14d8071e2d45451...	2016-10-03	2016-10-27	2016-11-07	24	-10
10	cd3b8574c82b42fc8129f6d502...	2016-10-03	2016-10-14	2016-11-23	10	-39

6. Analysis based on the payments:

1. Find the month on month no. of orders placed using different payment types.

```
SQL CODE : SELECT
  EXTRACT(YEAR FROM o.order_purchase_timestamp) AS
order_year,
  EXTRACT(MONTH FROM o.order_purchase_timestamp) AS
order_month,
  p.payment_type,
  COUNT(DISTINCT o.order_id) AS total_orders
FROM `Target.orders` o
JOIN `Target.payments` p
  ON o.order_id = p.order_id
GROUP BY order_year, order_month, p.payment_type
ORDER BY order_year, order_month, total_orders DESC
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

tow	order_year	order_month	payment_type	total_orders
1	2016	9	credit_card	3
2	2016	10	credit_card	253
3	2016	10	UPI	63
4	2016	10	voucher	11
5	2016	10	debit_card	2
6	2016	12	credit_card	1
7	2017	1	credit_card	582
8	2017	1	UPI	197
9	2017	1	voucher	33
10	2017	1	debit_card	9

2. Find the no. of orders placed on the basis of the payment installments that have been paid.

SQL CODE :

```
SELECT
  p.payment_installments,
  COUNT(DISTINCT p.order_id) AS total_orders
FROM `Target.payments` p
GROUP BY p.payment_installments
ORDER BY p.payment_installments
LIMIT 10;
```

Query results

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	payment_installm...	total_orders
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644